

ICS1512 Machine Learning Algorithms Laboratory

Experiment No. 02

Experiment Title: Email Spam/Ham Classification using Naïve Bayes and KNN

Student Name	Sharruk S
Register Number	3122247001061
Faculty	Dr. Poreddy Ajay Kumar Reddy
GitHub Repository	https://github.com/Sharruk/3122247001061
Submission Date	August 2, 2026

1 Aim and Objective

Aim: To build and evaluate Email Spam/Ham classifiers using Naïve Bayes (Gaussian, Multinomial and Bernoulli) and the K-Nearest Neighbours (KNN) algorithm on the Spambase dataset, and to compare their performance using standard classification metrics.

Objectives:

- Build spam classifiers using Naïve Bayes and KNN.
- Compare Gaussian, Multinomial and Bernoulli Naïve Bayes variants.
- Study the effect of varying k on KNN performance.
- Compare KDTree and BallTree search algorithms for KNN.
- Optimize KNN using GridSearchCV and RandomizedSearchCV.
- Evaluate computational complexity and execution time of all models.
- Evaluate model performance using 5-Fold Cross Validation.

2 Theory

2.1 Bayes' Theorem

Naïve Bayes classifiers are probabilistic classifiers based on Bayes' theorem, which describes the probability of a class C given a feature vector X in terms of the likelihood, prior and evidence:

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)} \quad (1)$$

where $P(C | X)$ is the posterior probability of class C given the observed features X , $P(X | C)$ is the likelihood of observing X given class C , $P(C)$ is the prior probability of class C , and $P(X)$ is the evidence (marginal probability of X).

The "naïve" assumption is that all features $x_1, x_2, \dots, x_n$ are conditionally independent given the class C , so that:

$$P(X | C) = \prod_{i=1}^n P(x_i | C) \quad (2)$$

The predicted class is the one that maximizes the posterior probability:

$$\hat{C} = \arg \max_C P(C) \prod_{i=1}^n P(x_i | C) \quad (3)$$

2.2 Gaussian Naïve Bayes

Gaussian Naïve Bayes (GNB) assumes that the continuous features associated with each class follow a normal (Gaussian) distribution. The likelihood of a feature x_i given class C is modelled as:

$$P(x_i | C) = \frac{1}{\sqrt{2\pi\sigma_C^2}} \exp\left(-\frac{(x_i - \mu_C)^2}{2\sigma_C^2}\right) \quad (4)$$

where μ_C and σ_C^2 are the mean and variance of feature x_i estimated from the training samples belonging to class C . GNB is well suited to real-valued, continuous features such as word-frequency ratios.

2.3 Multinomial Naïve Bayes

Multinomial Naïve Bayes (MNB) is designed for discrete count-based data, such as word frequencies in text classification. The likelihood of feature x_i (e.g., the frequency of a word) given class C is:

$$P(x_i | C) = \frac{N_{C,i} + \alpha}{N_C + \alpha n} \quad (5)$$

where $N_{C,i}$ is the count of feature i in class C , N_C is the total count of all features in class C , n is the number of features, and α is a Laplace/Lidstone smoothing parameter (typically $\alpha = 1$) used to avoid zero probabilities for unseen feature values.

2.4 Bernoulli Naïve Bayes

Bernoulli Naïve Bayes (BNB) models each feature as a binary variable indicating the presence or absence of a term, regardless of frequency. The likelihood is given by:

$$P(x_i | C) = p_{C,i}^{x_i} (1 - p_{C,i})^{(1-x_i)} \quad (6)$$

where $x_i \in \{0, 1\}$ indicates whether feature i is present, and $p_{C,i}$ is the probability that feature i occurs in class C . Unlike MNB, BNB explicitly penalizes the non-occurrence of a feature, which can be advantageous for short documents.

2.5 K-Nearest Neighbours (KNN)

KNN is a non-parametric, instance-based (lazy) learning algorithm. It does not build an explicit model during training; instead, it stores the training data and classifies a new sample based on the majority class among its k nearest neighbours, as measured by a distance metric. The predicted class is:

$$\hat{y} = \text{mode}(\{y_i : x_i \in N_k(x)\}) \quad (7)$$

where $N_k(x)$ is the set of k nearest training points to the query point x . The Euclidean and Manhattan distances used to determine neighbours are defined respectively as:

$$d_{\text{Euclidean}}(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (8)$$

$$d_{\text{Manhattan}}(x, y) = \sum_{i=1}^n |x_i - y_i| \quad (9)$$

Because KNN relies on distance computations, feature scaling (normalization/standardization) is essential prior to training.

2.6 KDTree

A KDTree (K-Dimensional Tree) is a space-partitioning binary tree data structure used to organize points in a k -dimensional space. It recursively splits the data along alternating axes, which allows nearest-neighbour queries to be answered in $O(\log n)$ average time instead of the $O(n)$ time required by a brute-force search, making it efficient for low to moderate dimensional feature spaces.

2.7 BallTree

A BallTree is another space-partitioning data structure that organizes points into a nested set of hyper-spheres (balls) rather than axis-aligned splits. It generally performs better than KDTree in higher-dimensional feature spaces because ball-shaped partitions handle the curse of dimensionality more gracefully than the axis-aligned splits used by a KDTree.

3 Dataset Description

The Spambase dataset was used for this experiment. It contains numerical features extracted from email messages, describing word frequencies, character frequencies, and statistics of capital-letter run-lengths, along with a binary target label indicating whether the email is spam or ham (not spam).

Property	Value
Dataset Name	Spambase
Dataset Source	https://www.kaggle.com/datasets/somesh24/spambase
Number of Samples	4601
Number of Features	57 (+ 1 target column, total 58 columns)
Number of Classes	2 (0 = Ham, 1 = Spam)
Missing Values	0
Duplicate Rows	391
Class Distribution	Ham: 2788 (60.6%), Spam: 1813 (39.4%)
Train-Test Split	Train: 3680 samples, Test: 921 samples (80:20)

The feature set consists of `word_freq_*` columns (frequency of specific words as a percentage of total words), `char_freq_*` columns (frequency of specific characters), and three columns describing the length of sequences of consecutive capital letters (`capital_run_length_average`, `capital_run_length_longest`, `capital_run_length_total`). The final column, `class`, is the binary target (1 = spam, 0 = ham).

4 Exploratory Data Analysis

The notebook performed the following exploratory steps on the Spambase dataset:

- Displayed the first 5 rows of the dataset to inspect the feature columns.
- Generated descriptive statistics (count, mean, std, min, quartiles, max) for all 57 numerical features using `df.describe()`.
- Verified missing values column-by-column using `df.isnull().sum()` – confirmed 0 missing values across all columns.
- Counted duplicate rows using `df.duplicated().sum()` – found 391 duplicate rows.
- Computed the class distribution (raw counts and percentages) of the target column.

Class Distribution (from notebook output):

Class	Count	Percentage
Ham (0)	2788	60.6%
Spam (1)	1813	39.4%

The classes are moderately imbalanced (roughly 61:39), which is not severe enough to require resampling techniques, but is relevant when interpreting precision, recall and F1-score.

Note on required EDA plots: The official experiment sheet requires a class distribution plot, a correlation heatmap, histograms of important features, and boxplots. My notebook (`ex2.ipynb`) computed the underlying statistics described above (class counts, descriptive statistics) but did not generate the class distribution plot, correlation heatmap, histogram, or boxplot figures as separate visual outputs. In accordance with the reporting requirement to use only my notebook's actual outputs, these four plots are stated here as: **Not generated in the notebook.**

5 Data Preprocessing

- **Missing Value Handling:** The Spambase dataset contained no missing values (confirmed via `df.isnull().sum()`), so no imputation or row removal was required.
- **Encoding:** The target column `class` is already a binary numeric label (0 = Ham, 1 = Spam), so no additional label encoding was necessary.
- **Feature Scaling:** All 57 numerical features were scaled using `MinMaxScaler` from `sklearn.preprocessing`, which rescales each feature to the range $[0, 1]$. Feature scaling is essential for KNN because the algorithm relies directly on distance computations between feature vectors.
- **Train-Test Split:** The dataset was split into training and test sets using `train_test_split` with `test_size=0.2`, `random_state=42`, and `stratify=y` to preserve the class ratio in both splits. This produced:
 - `X_train`: (3680, 57)
 - `X_test`: (921, 57)
 - `y_train`: (3680,)

- `y_test`: (921,)
- **Feature Engineering:** No additional feature engineering was applied, as the dataset already contains pre-extracted numerical features (word frequencies, character frequencies, and capital-run-length statistics).

6 Experimental Setup

The following details were detected directly from the notebook file (`ex2.ipynb`) metadata and source code:

Parameter	Value
Python Version	3.13.13 (from notebook kernel metadata)
IDE / Kernel	Jupyter Notebook (<code>python3</code> kernel)
Core Libraries Used	NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn
Scikit-learn Modules Used	GaussianNB, MultinomialNB, BernoulliNB, KNeighborsClassifier, MinMaxScaler, GridSearchCV, RandomizedSearchCV, <code>cross_val_score</code>
Operating System / Hardware	Not specified in the notebook
Library Version Numbers (e.g. scikit-learn)	Not printed in the notebook output

Only the Python version could be reliably detected, since it is recorded in the notebook's `language_info` metadata. Operating system, hardware, and exact library version numbers were not printed anywhere in the notebook execution, and are therefore not fabricated here.

7 Hyperparameter Tuning

7.1 KNN Accuracy vs. k

KNN was evaluated for $k = 1$ to 10 in the notebook. The requested comparison values for $k = 1, 3, 5, 7, 9$ are reported below; $k = 11$ was outside the range tested in the notebook (`k_values = range(1, 11)` only covers $k = 1$ to $k = 10$).

k	Accuracy	Precision	Recall	F1-score
1	0.8958	0.8678	0.8678	0.8678
3	0.8958	0.8825	0.8485	0.8652
5	0.9023	0.8760	0.8760	0.8760
7	0.8958	0.8761	0.8567	0.8663
9	0.8947	0.8866	0.8402	0.8628
11	Not generated in the notebook			

The best accuracy among the tested values of k was obtained at $k = 5$ (Accuracy = 0.9023), which was selected by the notebook as `best_k`.

7.2 GridSearchCV vs. RandomizedSearchCV

Both search strategies were used to tune the `n_neighbors`, `weights` and `metric` hyperparameters of KNN over 5-fold cross validation.

Parameter	GridSearchCV	RandomizedSearchCV
Best k	8	5
Metric	manhattan	manhattan
Weights	distance	distance
Algorithm	Not tuned (<code>algorithm</code> not in param grid)	Not tuned (<code>algorithm</code> not in param grid)
CV Accuracy	0.9196	0.9158
Execution Time	32.8716 s	6.4397 s

GridSearchCV searched the full grid of $20 \times 2 \times 3 = 120$ combinations, while RandomizedSearchCV sampled only 20 random combinations (`n_iter=20`) from a larger space of $30 \times 2 \times 3 = 180$ combinations. GridSearchCV achieved a marginally higher cross-validation accuracy (0.9196 vs. 0.9158) but required roughly $5.1 \times$ longer execution time (32.87s vs. 6.44s).

7.3 KDTree vs. BallTree

Using the best k found from the accuracy-vs- k sweep ($k = 5$), the notebook compared four neighbour-search algorithms (`auto`, `ball_tree`, `kd_tree`, `brute`). The KDTree and BallTree rows relevant to this comparison are:

Accuracy	0.9023	0.9023
Training Time (s)	0.034315	0.029319
Prediction Time (s)	0.465262	0.579599

Both tree-based algorithms achieved identical accuracy (0.9023) to the brute-force and auto settings, since all methods search for the exact same k nearest neighbours – only the search strategy differs, not the classification result. BallTree trained marginally faster than KDTree (0.0293s vs. 0.0343s), while KDTree predicted faster than BallTree (0.4653s vs. 0.5796s). Notably, the plain `brute` force algorithm was fastest overall for both training (0.0019s) and prediction (0.0403s) on this dataset, since with 57 features the tree-based partitioning provides limited benefit over direct brute-force distance computation.

8 Performance Evaluation

8.1 Naïve Bayes Comparison

All three Naïve Bayes variants were trained on the same 80:20 train-test split and evaluated on the held-out test set (921 samples).

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.8328	0.8936	0.8795
Precision	0.7146	0.9431	0.8684
Recall	0.9587	0.7769	0.8182
F1-score	0.8188	0.8520	0.8426
ROC-AUC	0.9229	0.9625	0.9457

Multinomial Naïve Bayes achieved the highest accuracy (0.8936), precision (0.9431), F1-score (0.8520) and ROC-AUC (0.9625) among the three Naïve Bayes variants. Gaussian Naïve Bayes achieved the highest recall (0.9587) but the lowest precision (0.7146), indicating it aggressively flags emails as spam at the cost of more false positives. Bernoulli Naïve Bayes falls between the two, with balanced precision and recall.

8.2 Overall Model Comparison (including KNN)

The final comparison, computed after hyperparameter tuning identified the best KNN configuration ($k = 5$), is:

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
KNN ($k=5$)	0.9023	0.8760	0.8760	0.8760	0.9419
Multinomial NB	0.8936	0.9431	0.7769	0.8520	0.9625
Bernoulli NB	0.8795	0.8684	0.8182	0.8426	0.9457
Gaussian NB	0.8328	0.7146	0.9587	0.8188	0.9229

KNN with $k = 5$ achieved the highest overall test-set accuracy (0.9023) and F1-score (0.8760) among all four models, although Multinomial NB retained the highest ROC-AUC (0.9625), indicating it ranks spam vs. ham probabilities slightly better even though its default 0.5 decision threshold produced a lower F1-score than KNN.

9 Complexity Analysis

The theoretical time complexity of each algorithm, as specified in the official experiment sheet, is summarized below. Here n denotes the number of training samples and d the number of features.

Algorithm	Training	Prediction
Naïve Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)$ (average)
BallTree	$O(n \log n)$	$O(\log n)$ (average)

Interpretation:

- Naïve Bayes has linear training cost in the number of samples and features ($O(nd)$), since it only needs to estimate per-feature statistics (means/variances for Gaussian NB, or counts/probabilities for Multinomial/Bernoulli NB) once. Prediction is very fast, $O(d)$, requiring only a pass over the features for a single query point.

- Brute-force KNN has effectively no training cost ($O(1)$, since it just stores the data) but expensive prediction, $O(nd)$, because every query must compute its distance to all n training points across d dimensions.
- KDTree and BallTree incur an $O(n \log n)$ tree-construction cost during training but reduce the average prediction cost to $O(\log n)$ by pruning large portions of the search space – at the cost of degrading toward brute-force performance in high-dimensional spaces (the well-known “curse of dimensionality”).

10 Experimental Time Analysis

Actual training and prediction times were measured using Python’s `time.time()` around each model’s `fit()` and `predict()` calls, on the test set of 921 samples.

Algorithm	Training Time (s)	Prediction Time (s)
Gaussian NB	0.011095	0.003952
Multinomial NB	0.005796	0.000686
Bernoulli NB	0.010884	0.002240
Best KNN (k=5)	0.001862	0.060918

Comparison with theory: The measured results agree qualitatively with the theoretical complexity analysis. KNN had the lowest training time (0.0019s) since it performs no real “training” beyond storing the data ($O(1)$), which matches theory. However, KNN had by far the highest prediction time (0.0609s) – roughly 16 to 89 times slower than the Naïve Bayes variants – because it must compute distances to training points for every test query ($O(nd)$), whereas Naïve Bayes predictions only require a single pass over the feature vector ($O(d)$) using pre-computed class statistics. Among the Naïve Bayes variants, Multinomial NB was fastest for both training and prediction, consistent with its simpler count-based probability updates compared to Gaussian NB’s per-feature mean/variance computations.

11 Results

11.1 5-Fold Cross Validation

The notebook computed `cross_val_score` with `cv=5` for each model on the full (scaled) dataset, but only stored and displayed the aggregated mean and standard deviation of the fold scores (`cv_df`); the individual per-fold accuracy values were not printed or stored separately in the notebook’s output. In line with the rule to use only actual notebook outputs, the individual fold rows below are marked accordingly, while the Average row reflects the real `cv_df` output.

Fold	Gaussian NB	Multinomial NB	Bernoulli NB	Best KNN (k=5)
1		Not generated in the notebook		
2		Not generated in the notebook		
3		Not generated in the notebook		
4		Not generated in the notebook		
5		Not generated in the notebook		
Average (Std. Dev.)	0.8222 (0.0637)	0.8687 (0.0245)	0.8809 (0.0531)	0.8646 (0.0573)

Under 5-fold cross validation across the full dataset, Bernoulli Naïve Bayes achieved the highest mean accuracy (0.8809), followed by KNN (0.8646), Multinomial NB (0.8687 – lowest variance at 0.0245), and Gaussian NB (0.8222, also the highest variance at 0.0637). This differs from the single 80:20 test-split ranking (Section 8) where KNN and Multinomial NB led, showing that model ranking can shift somewhat depending on the evaluation protocol used.

11.2 Figures

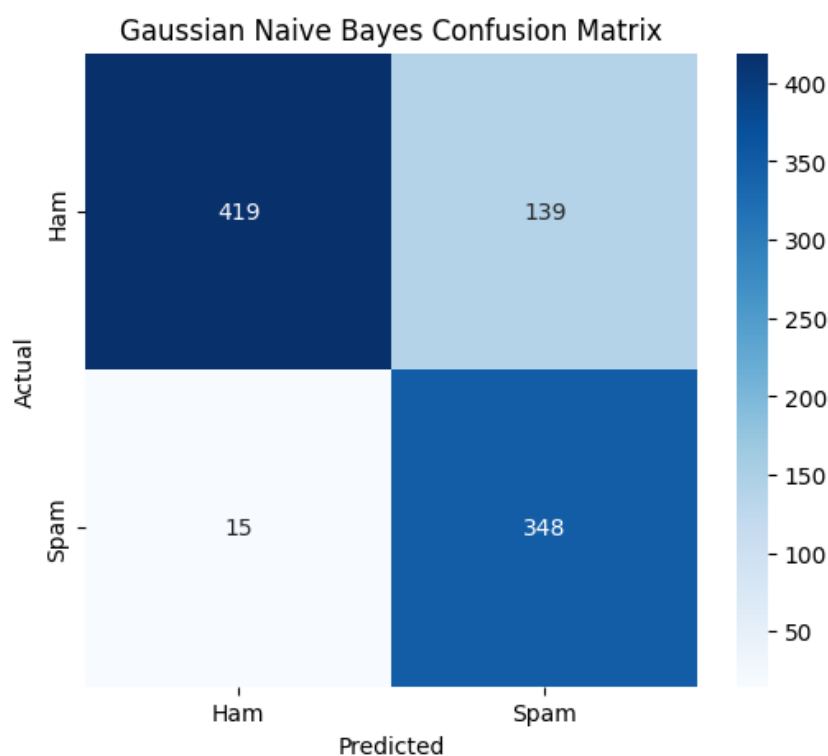


Figure 1: Confusion Matrix – Gaussian Naïve Bayes (test set, 921 samples)

Inference: The Gaussian NB confusion matrix shows a strong tendency toward false positives: many ham emails are misclassified as spam, consistent with its low precision (0.7146). Recall for the spam class is very high (0.9587), meaning it rarely misses actual spam. This trade-off explains why Gaussian NB is the most "aggressive" spam filter of the three NB variants, at the cost of flagging legitimate emails.

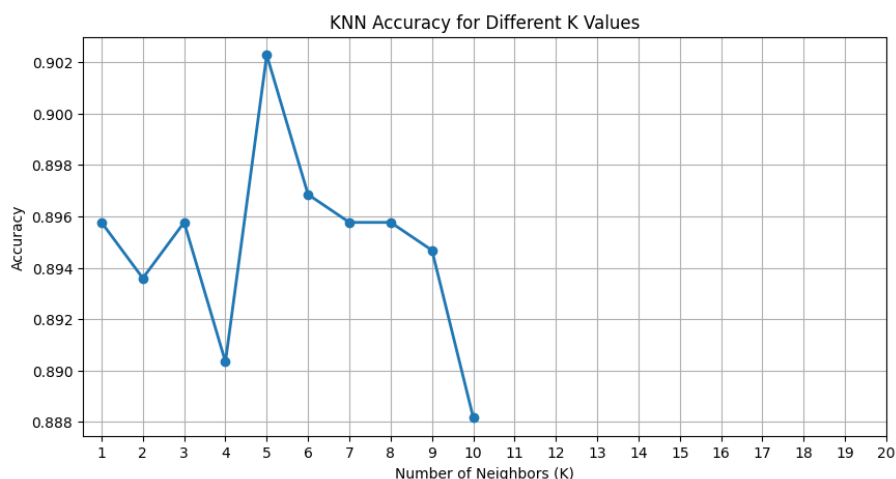


Figure 2: KNN Accuracy for Different Values of k ($k = 1$ to 10)

Inference: Accuracy fluctuates across the tested range of k , peaking at $k = 5$ (0.9023). Very small k (e.g. $k = 1$) can overfit to noise in individual neighbours, while larger k values begin to smooth the decision boundary, occasionally trading a small amount of accuracy for stability. The curve does not show a single monotonic trend, indicating that k around 5-9 is a reasonable operating range for this dataset.

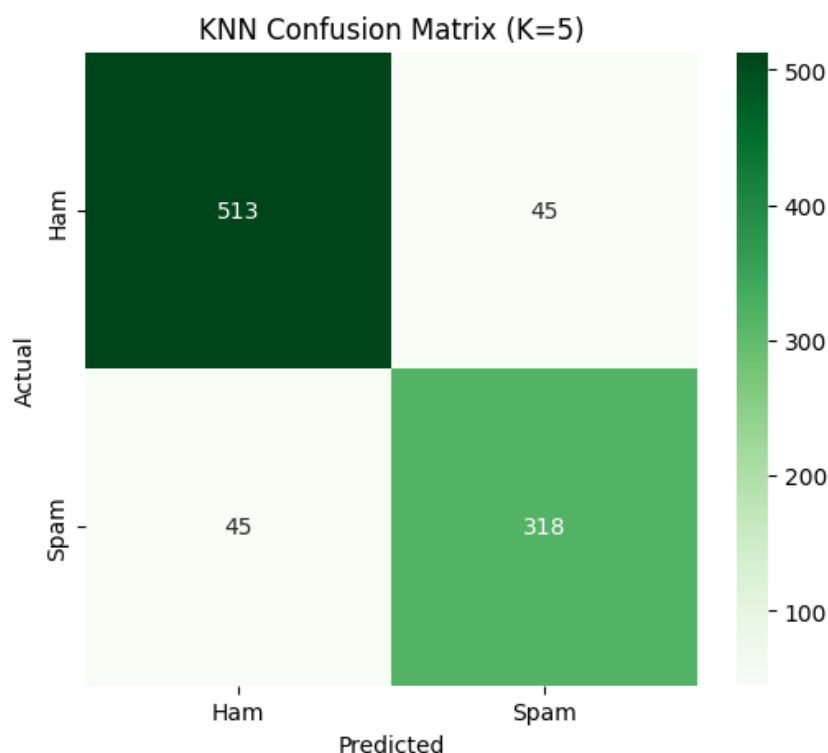


Figure 3: Confusion Matrix – KNN with Best $k = 5$

Inference: At the best-performing $k = 5$, KNN shows a more balanced confusion matrix than Gaussian NB, with similar precision (0.92) and recall (0.92) for the ham class and precision (0.88)/recall (0.88) for the spam class. This balance is reflected in its equal precision, recall and F1-score of 0.876 for the spam class, making $k = 5$ KNN the most balanced classifier evaluated

on the single test split.

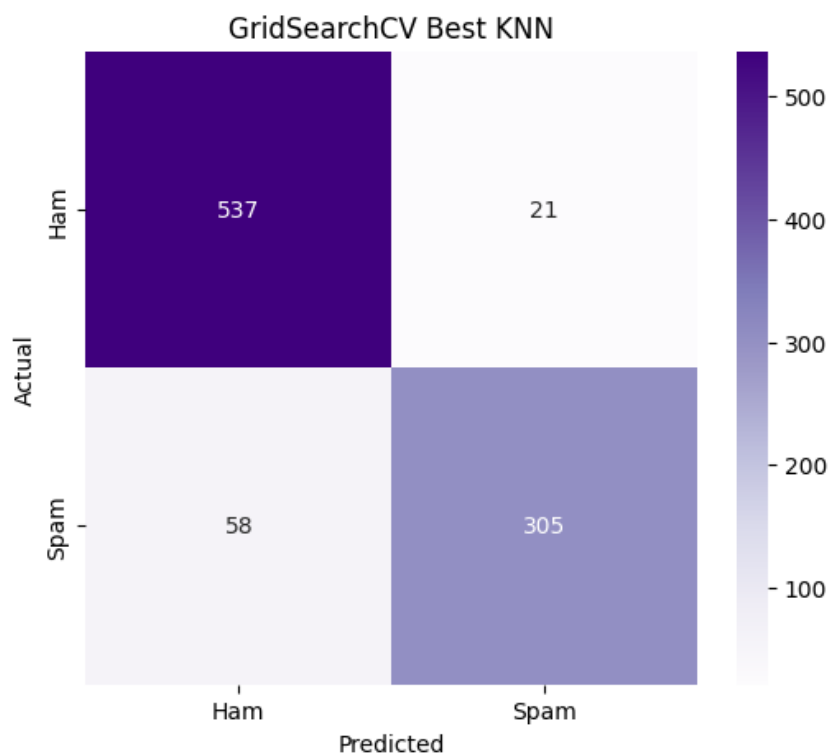


Figure 4: Confusion Matrix – GridSearchCV Best KNN ($k=8$, manhattan, distance-weighted)

Inference: The GridSearchCV-tuned KNN model (Accuracy 0.9142) improves precision for the spam class considerably (0.9356) compared to the untuned $k = 5$ model, at some cost to recall (0.8402). This indicates that tuning distance metric and weighting shifted the model toward being more conservative when labelling an email as spam, reducing false positives.

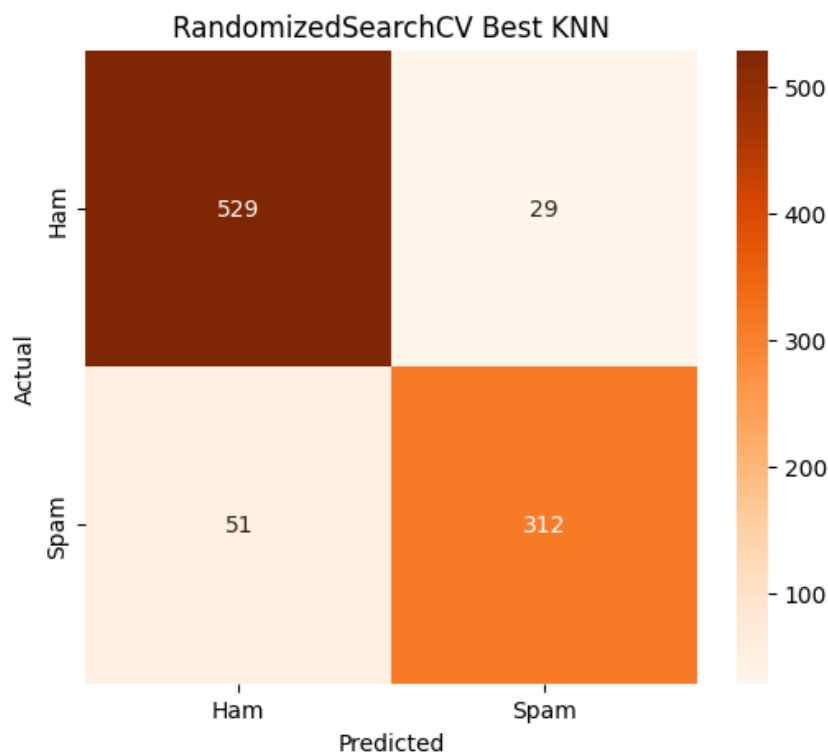


Figure 5: Confusion Matrix – RandomizedSearchCV Best KNN (k=5, manhattan, distance-weighted)

Inference: RandomizedSearchCV converged on $k = 5$ with the manhattan metric and distance weighting, achieving accuracy 0.9131 – very close to GridSearchCV’s 0.9142 despite exploring far fewer parameter combinations (20 vs. 120). This confirms that RandomizedSearchCV can find a near-optimal configuration at a fraction of the computational cost.

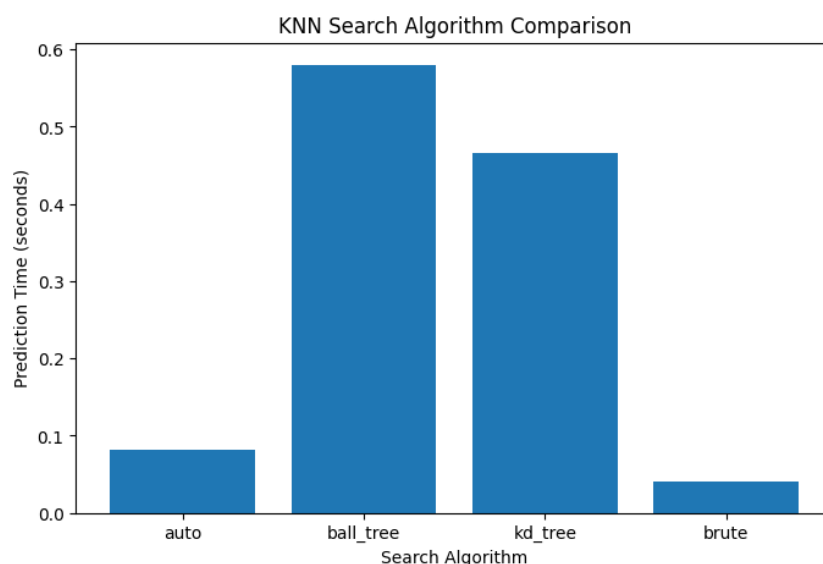


Figure 6: KNN Search-Algorithm Comparison: Prediction Time by Algorithm (auto, ball_tree, kd_tree, brute)

Inference: The `brute` force algorithm has the lowest prediction time, while `ball_tree` and

`kd_tree` show substantially higher prediction times in this experiment. This is a direct consequence of the dataset's dimensionality (57 features): tree-based structures lose their pruning advantage in higher-dimensional spaces, so the overhead of traversing the tree outweighs the benefit, making brute-force competitive or even faster here.

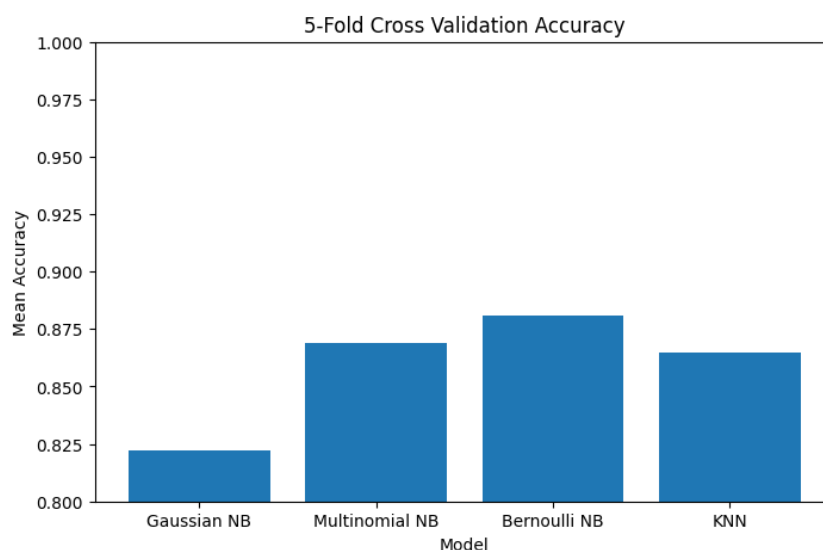


Figure 7: 5-Fold Cross Validation Mean Accuracy by Model

Inference: The bar chart shows Bernoulli NB with the highest mean cross-validation accuracy, followed closely by KNN and Multinomial NB, with Gaussian NB clearly the lowest. This confirms that under repeated 5-fold resampling, Bernoulli NB generalizes at least as well as the tuned KNN model, even though KNN edged ahead on the single 80:20 held-out split.

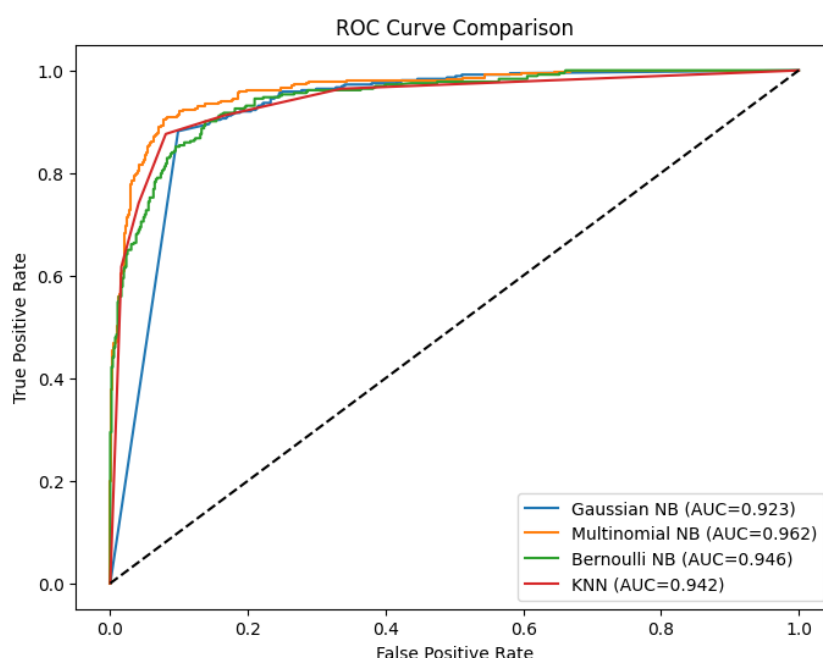


Figure 8: ROC Curve Comparison – Gaussian NB, Multinomial NB, Bernoulli NB, and Best KNN

Inference: All four curves lie well above the diagonal (random-guess) line, confirming strong

discriminative ability. Multinomial NB has the highest AUC (0.962), followed by Bernoulli NB (0.946) and KNN (0.942), with Gaussian NB lowest (0.923). This shows that although KNN had the best accuracy at a fixed threshold, Multinomial NB is the best overall ranker of spam-vs-ham probabilities across all thresholds.

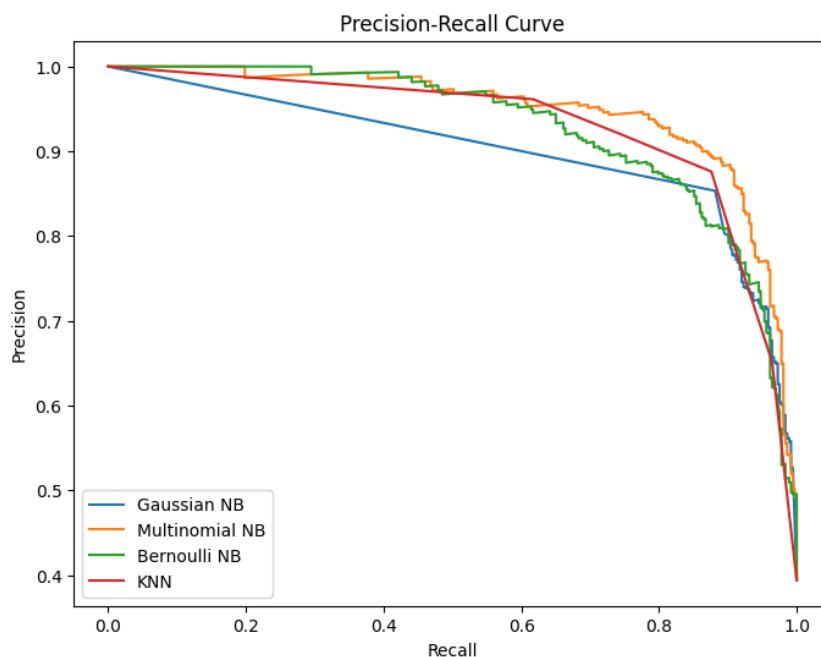


Figure 9: Precision-Recall Curve Comparison

Inference: The precision-recall curves reinforce the ROC findings: Multinomial NB maintains high precision across a wide range of recall values, while Gaussian NB's curve drops off more steeply at higher recall, reflecting its lower precision at the default threshold. This view is particularly informative here since the classes are moderately imbalanced (60.6% ham vs. 39.4% spam).

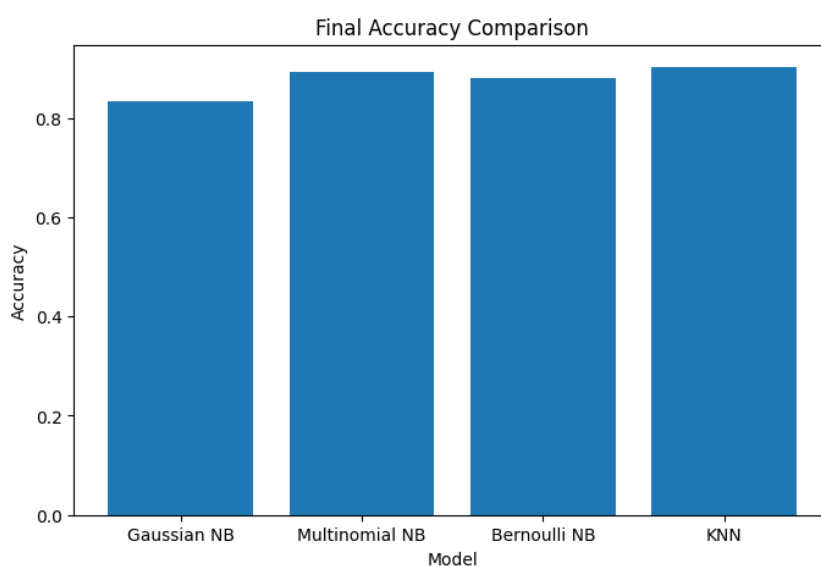


Figure 10: Final Accuracy Comparison Across All Four Models

Inference: The final bar chart confirms that KNN ($k=5$) achieved the highest test accuracy (0.9023), ahead of Multinomial NB (0.8936), Bernoulli NB (0.8795), and Gaussian NB (0.8328). This chart summarizes the overall ranking of models on the single held-out test split used throughout the experiment.

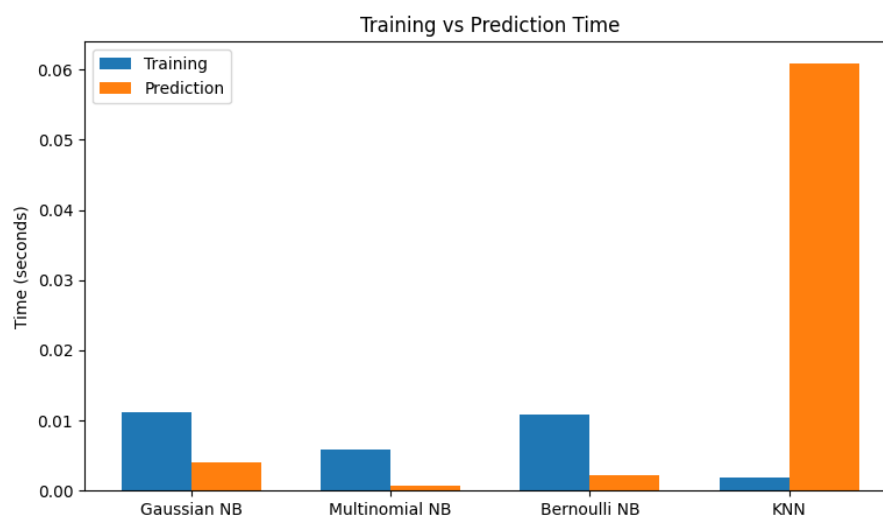


Figure 11: Training Time vs. Prediction Time Comparison Across All Four Models

Inference: KNN clearly stands out with negligible training time but the highest prediction time among all four models, visually confirming the theoretical trade-off discussed in Section 10. Multinomial NB and Bernoulli NB show the fastest overall prediction times, making them attractive choices when prediction latency is critical (e.g., real-time spam filtering).

11.3 Note on Missing Plots

The official experiment sheet also requires the following plots. These were not generated as separate figures in my notebook (`ex2.ipynb`), and are therefore reported here as **Not generated in the notebook**, rather than being fabricated:

- Class distribution plot (only text-based value counts were computed).
- Correlation heatmap.
- Histograms of important features.
- Boxplots.
- Individual confusion matrices for Multinomial NB and Bernoulli NB (only Gaussian NB, KNN $k=5$, GridSearchCV-KNN, and RandomizedSearchCV-KNN confusion matrices were plotted).
- GridSearchCV heatmap (only `best_params_` and `best_score_` were printed as text).
- RandomizedSearchCV score distribution (only `best_params_` and `best_score_` were printed as text).

12 Discussion

This section answers the analysis questions posed in the official experiment sheet, based strictly on the results obtained in my notebook.

1. Which Naïve Bayes variant performed best? On the single 80:20 test split, Multinomial

Naïve Bayes performed best overall, with the highest accuracy (0.8936), precision (0.9431), F1-score (0.8520) and ROC-AUC (0.9625) among the three variants. However, under 5-fold cross validation on the full dataset, Bernoulli Naïve Bayes had the highest mean accuracy (0.8809) with a lower standard deviation than Gaussian NB. Overall, Multinomial NB is the strongest choice when ranking quality (ROC-AUC) and precision matter most, while Bernoulli NB is most consistent across cross-validation folds.

2. What is the optimal value of k ? Sweeping k from 1 to 10 showed the best test-set accuracy (0.9023) at $k = 5$. RandomizedSearchCV independently confirmed $k = 5$ (with the manhattan metric and distance weighting) as its best configuration, while GridSearchCV selected a slightly larger $k = 8$ (also manhattan/distance) with a marginally higher cross-validation accuracy (0.9196 vs. 0.9158). Taken together, k in the range of 5-8 with the manhattan distance metric and distance-based weighting is optimal for this dataset.

3. Compare GridSearchCV and RandomizedSearchCV. GridSearchCV exhaustively evaluated all 120 combinations of `n_neighbors` (1-20), `weights`, and `metric`, taking 32.87 seconds and achieving a cross-validation accuracy of 0.9196. RandomizedSearchCV sampled only 20 combinations from a larger space of 180 (`n_neighbors` 1-30), taking just 6.44 seconds and achieving 0.9158 cross-validation accuracy – about $5.1\times$ faster for a difference of only 0.0038 in accuracy. This illustrates the classic exhaustive-search vs. sampled-search trade-off: RandomizedSearchCV is far more efficient with only a small penalty in the quality of the discovered hyperparameters.

4. Compare KDTree and BallTree. Both KDTree and BallTree achieved identical classification accuracy (0.9023) to the brute-force baseline at $k = 5$, since the choice of search structure does not change which neighbours are found – only how efficiently they are found. In terms of speed, BallTree trained marginally faster (0.0293s vs. 0.0343s) while KDTree predicted faster (0.4653s vs. 0.5796s). Interestingly, plain brute-force search was faster than both tree structures for prediction on this dataset (0.0403s), because the 57-dimensional feature space is high enough that tree-based pruning provides diminishing returns – a known limitation of both KDTree and BallTree in higher dimensions.

5. Compare theoretical and practical complexity. The measured times were broadly consistent with theory. KNN's training time was the smallest in practice (0.0019s), matching its theoretical $O(1)$ training complexity, while its prediction time was by far the largest (0.0609s), consistent with its theoretical $O(nd)$ prediction cost. Naïve Bayes variants showed the opposite pattern – modest training time ($O(nd)$ in theory) and very fast prediction ($O(d)$ in theory), which matched the observed prediction times of under 0.004 seconds for all three variants. The KDTree/BallTree results, however, show that the theoretical average-case $O(\log n)$ prediction advantage over brute force did not materialize in practice at $d = 57$ features, underscoring that theoretical complexity bounds assume favourable (typically low-dimensional) conditions that do not always hold for real datasets.

6. Which classifier is preferred for large datasets? Based on both theoretical complexity and the measured experimental times, Naïve Bayes (particularly Multinomial NB, which had the fastest measured training and prediction times of all four models) is preferable for large datasets. Its $O(nd)$ training and $O(d)$ prediction complexity scale far better than KNN's $O(nd)$ prediction cost, which grows directly with the number of training samples n and becomes a bottleneck as the dataset grows, especially in applications requiring low-latency predictions such as real-time spam filtering.

13 Conclusion

This experiment implemented and compared Email Spam/Ham classification using three Naïve Bayes variants (Gaussian, Multinomial, Bernoulli) and the K-Nearest Neighbours algorithm on the Spambase dataset (4601 samples, 57 features). After MinMax feature scaling and an 80:20 stratified train-test split, Multinomial Naïve Bayes emerged as the strongest Naïve Bayes variant, achieving 89.36% accuracy and the highest ROC-AUC (0.9625) among the three variants. Hyperparameter tuning of KNN via GridSearchCV and RandomizedSearchCV both converged on the manhattan distance metric with distance-based weighting, at $k = 8$ and $k = 5$ respectively, with GridSearchCV obtaining a marginally higher cross-validation accuracy (0.9196) at roughly $5\times$ the computational cost of RandomizedSearchCV (0.9158). On the held-out test set, KNN with $k = 5$ achieved the highest overall accuracy (90.23%) and F1-score (0.876) among all four models, while Multinomial NB retained the best ROC-AUC. Under 5-fold cross validation on the full dataset, however, Bernoulli NB proved the most consistent performer (mean accuracy 0.8809). KDTree and BallTree produced identical accuracy to brute-force KNN, differing only in computational speed, and in this 57-dimensional feature space, brute-force search was in fact the fastest neighbour-search strategy. Overall, the experiment demonstrates the classic accuracy-versus-latency trade-off between Naïve Bayes (fast, scalable, slightly lower peak accuracy) and KNN (higher peak accuracy, but expensive prediction cost that scales with dataset size), and confirms that hyperparameter tuning and cross-validation are essential tools for selecting a robust, well-generalizing spam classifier.

14 References

1. Scikit-learn Documentation. Available: <https://scikit-learn.org>
2. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 3rd ed., O'Reilly Media, 2022.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. Kaggle Spambase Dataset. Available: <https://www.kaggle.com/datasets/somesh24/spambase>